Penetration Testing Report

Full Name: Egeonu Chioma Lilian
Program: HCS - Penetration Testing Internship Week-2
Date:20/02/2025

Introduction

This report document hereby describes the proceedings and results of a Black Box security assessment conducted against the **Week 2 Labs**. The report hereby lists the findings and corresponding best practice mitigation actions and recommendations.

1. Objective

The objective of the assessment was to uncover vulnerabilities in the **Week 2 Labs** and provide a final security assessment report comprising vulnerabilities, remediation strategy and recommendation guidelines to help mitigate the identified vulnerabilities and risks during the activity.

2. Scope

This section defines the scope and boundaries of the project.

Application Name	**SQL injection, Insecure Direct Object References**

3. Summary

Outlined is a Black Box Application Security assessment for the **Week 2 Labs**.

Total number of Sub-labs: 16 Sub-labs

High	Medium	Low
4	6	6

High - **Number of Sub-labs with hard difficulty level**

Medium - **Number of Sub-labs with Medium difficulty level**

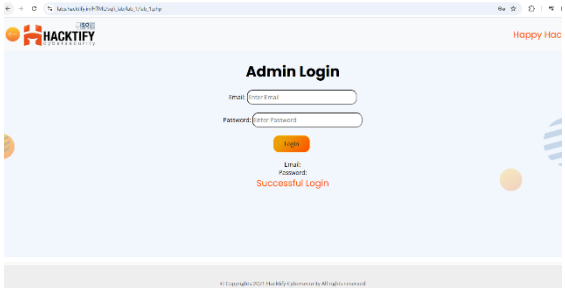
Low - **Number of Sub-labs with Easy difficulty level**

1. SQL Injection

1.1. Strings and Errors Part 1!

Reference	Risk Rating
Strings and Errors Part 1!	**Low**
Tools Used	
Manual.	
Vulnerability Description	
Error-based SQL injection is a type of security vulnerability and attack that occurs when an attacker injects malicious SQL statements into a web application's input fields, causing the application to generate SQL errors. These errors can reveal sensitive information about the application's database structure, data, or configuration.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_1/lab_1.php	
Consequences of not Fixing the Issue	
Unauthorized data access, Data manipulation and information leakage.	
Suggested Countermeasures	
Input validation and sanitization, Parameterized queries or prepared statements, Least privilege principle amongst others.	
References	
https://beaglesecurity.com/blog/vulnerability/error-based-sqli.html https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_for_SQL_Injection	

Proof of Concept

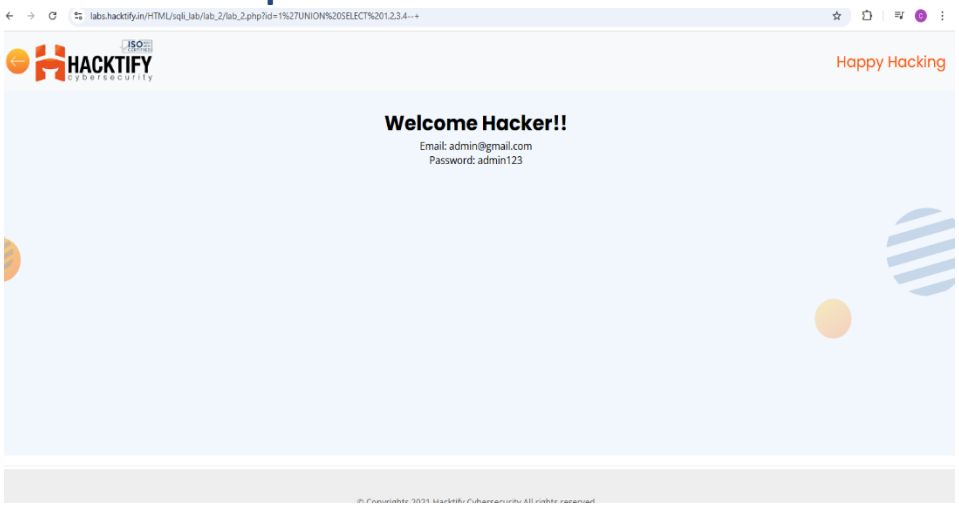


1.2. Strings and Errors Part 2!

Reference	Risk Rating
Strings and Errors Part 2!	**Low**
Tools Used	
Manual analysis without specific tools.	
Vulnerability Description	

SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. This can allow an attacker to view data that they are not normally able to retrieve. This might include data that belongs to other users, or any other data that the application can access. In many cases, an attacker can modify or delete this data, causing persistent changes to the application's content or behavior.
How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/sqli_lab/lab_2/index.php
Consequences of not Fixing the Issue
Passwords and information leakage, Credit card details and Personal user information.
Suggested Countermeasures
Implement strict input validation and output encoding to sanitize user-supplied data and prevent script injection.
References
https://portswigger.net/web-security/sql-injection

Proof of Concept

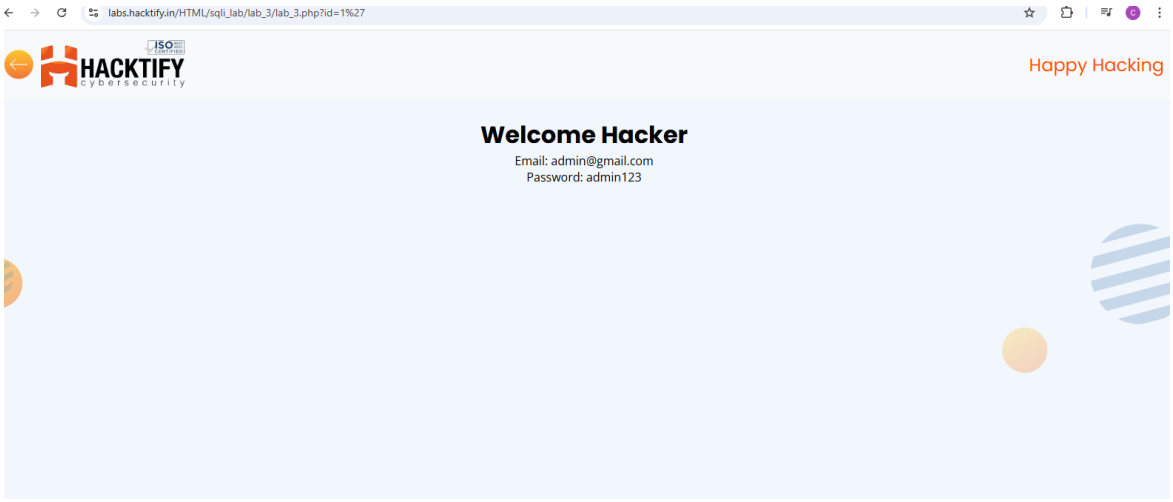


1.3. Strings and Errors Part 3!

Reference	Risk Rating
Strings and Errors Part 3!	**Low**
Tools Used	
Manual analysis without specific tools.	
Vulnerability Description	
the attacker performs actions that cause the database to produce error messages. The attacker can potentially use the data provided by these error messages to gather information about the structure of the database.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	

https://labs.hacktify.in/HTML/sqli_lab/lab_3/index.php
Consequences of not Fixing the Issue
Financial loss, Data manipulation and leakage, legal repercussions and reputational damage.
Suggested Countermeasures
Input data validation, always **Sanitize and Validate All User Inputs, Use Prepared Statements and Parameterized Queries**.
References
https://www.sentinelone.com/cybersecurity-101/cybersecurity/sql-injection/#:~:text=Poor%20Input%20Validation%2C%20Weak%20Query,issues%2C%20and%20even%20legal%20repercussions. https://portswigger.net/web-security/sql-injection/blind/lab-sql-injection-visible-error-based

Proof of Concept

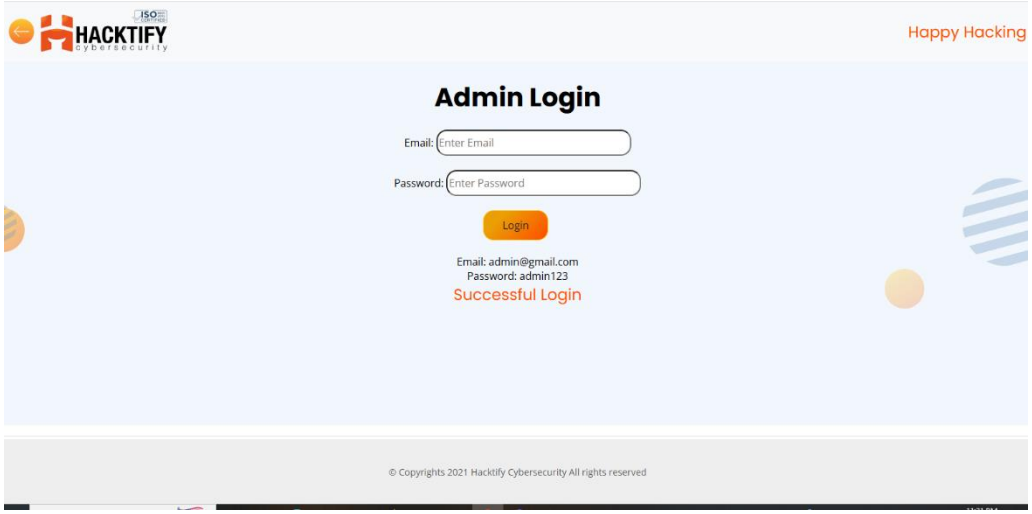


1.4. Let's Trick 'em!

Reference	Risk Rating
Let's Trick 'em!	**Medium**
Tools Used	
No tools are used.	
Vulnerability Description	
The application exhibited a critical vulnerability allowing unauthorized access to sensitive user data through simple SQL injection attacks. By manipulating input parameters within GET requests, an attacker could retrieve a comprehensive list of sensitive information. This indicates a severe lack of input validation and sanitization on the application's side.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_4/lab_4.php	
Consequences of not Fixing the Issue	

A malicious actor with SQL injection knowledge can exploit vulnerabilities in web applications to extract sensitive user data. By manipulating input parameters, attackers can construct malicious SQL queries that bypass application controls and directly access the database. This enables them to retrieve confidential information such as credit card numbers, email addresses, names, phone numbers, and physical addresses, posing a significant risk to user privacy and data security..
Suggested Countermeasures
Use Stored Procedure, Not Dynamic SQL, Use Object Relational Mapping (ORM) Framework, Least Privilege, Input Validation, Character Escaping and Use WAF (Web Application Firewall).
References
https://medium.com/@muhammad22/sql-injection-by-injecting-queries-in-the-url-get-parameter-951e65b7fe98 https://www.imperva.com/learn/application-security/sql-injection-sqli/#:~:text=SQL%20injections%20typically%20fall%20under,data%20and%20their%20damage%20potential.

Proof of Concept

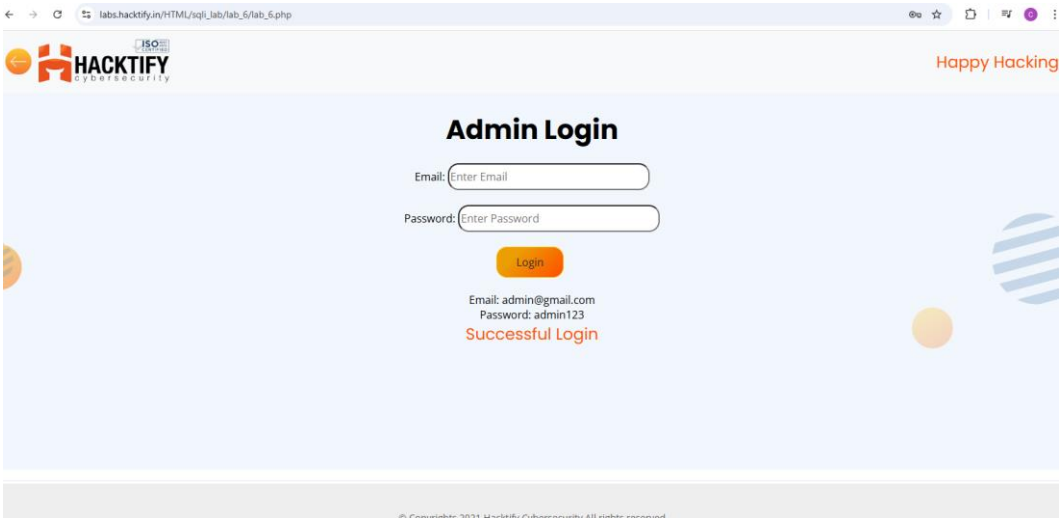


1.5. Booleans and Blind!

Reference	Risk Rating
Booleans and Blind!	**High**
Tools Used	
Manual(website inspection and observation)	
Vulnerability Description	
Boolean-based blind SQL injection Vulnerability is a subtype of blind SQL injection where the attacker observes the behavior of the database server and the application after combining legitimate queries with malicious data using Boolean operators.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_5/index.php	
Consequences of not Fixing the Issue	

Data and information leakage.
Suggested Countermeasures
Use of parameterized queries (also known as prepared statements) to access SQL databases
References
https://www.invicti.com/learn/blind-sql-injection/

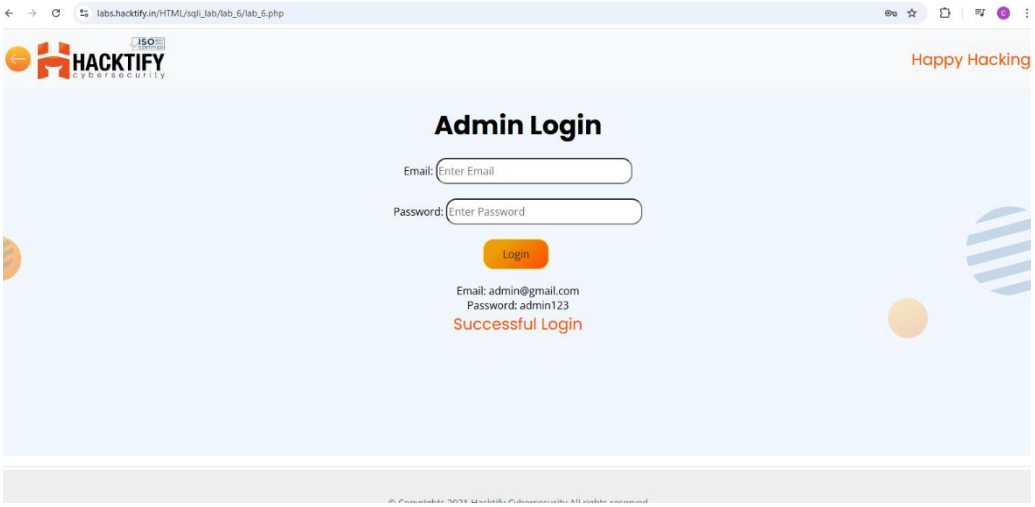
Proof of Concept



1.6. Error Based: Tricked!

Reference	Risk Rating
Error Based: Tricked!	**Medium**
Tools Used	
Manual analysis without any specific tools.	
Vulnerability Description	
Error-based SQLi is an in-band SQL Injection technique that relies on error messages thrown by the database server to obtain information about the structure of the database. In some cases, error-based SQL injection alone is enough for an attacker to enumerate an entire database. While errors are very useful during the development phase of a web application, they should be disabled on a live site, or logged to a file with restricted access instead..	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_6/index.php	
Consequences of not Fixing the Issue	
Database Enumeration and Data Exfiltration.	
Suggested Countermeasures	
Input data validation and sanitization and use of parametrized queries.	
References	
https://www.acunetix.com/websitesecurity/sql-injection2/	

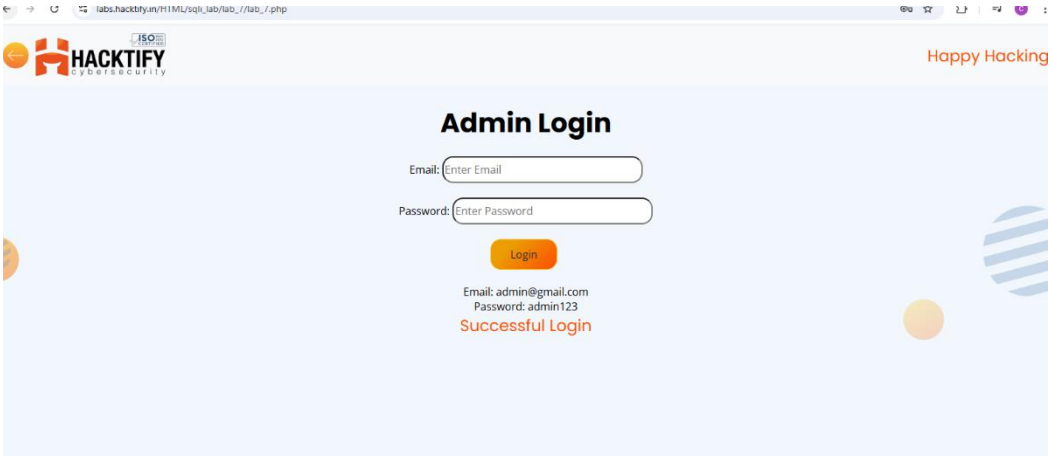
Proof of Concept



1.7. Errors and Post!

Reference	Risk Rating
Errors and Post!	**low**
Tools Used	
No tools are used.	
Vulnerability Description	
Error-based SQL injection is an In-band injection technique that enables threat actors to exploit error output from the database to manipulate its data. It manipulates the database into generating an error that informs the actor of the database's structure. This error-based SQL injection vulnerability, the attacker tries to insert a malicious query with the goal of receiving an error message that provides sensitive information about the database.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_7/index.php	
Consequences of not Fixing the Issue	
Suggested Countermeasures	
Use Dynamic Application Security Testing (DAST), Stored procedures are another way to prevent SQL injection. When used correctly, stored procedures can completely eliminate SQL injection vulnerabilities and implementation of principle of least privilege	
References	
https://brightsec.com/blog/error-based-sql-injection/	

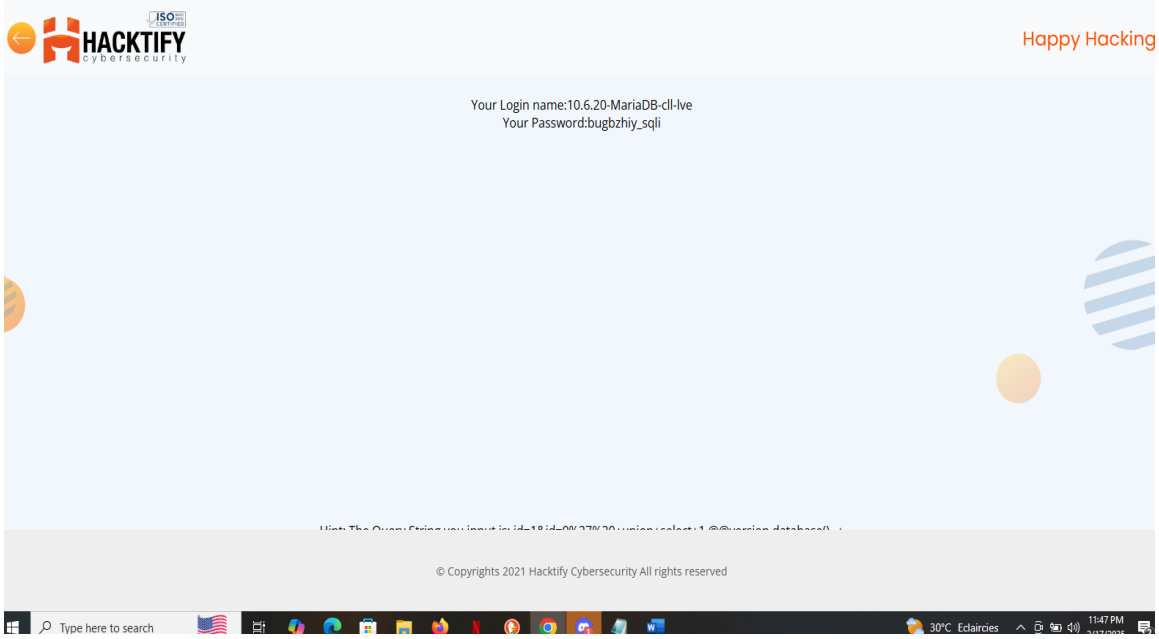
Proof of Concept



1.8. WAF's are injected!

Reference	Risk Rating
WAF's are injected	**High**
Tools Used	
Manual.	
Vulnerability Description	
In this type of vulnerability an attacker uses various methods to bypass WAFs. This method of bypassing WAF for SQL injection attacks uses JSON-based SQL commands.	
How It Was Discovered	
Discovered through manual analysis by intercepting and modifying requests using Burp Suite.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/xss_lab/lab_8/index.php	
Consequences of not Fixing the Issue	
Data manipulation, Authentication bypass, denial of service and data breach	
Suggested Countermeasures	
Input Validation: Strictly validate and sanitize all user inputs. Parameterized Queries: Use prepared statements and parameterized queries to ensure that user inputs are treated as data, not executable code. Stored Procedures: Use stored procedures that encapsulate SQL statements. ORMs (Object-Relational Mappers): Use ORM libraries that handle SQL queries securely. Escaping: Properly escape all user inputs to prevent them from being interpreted as SQL code. Least Privilege: Limit database user permissions to the minimum necessary for the application to function. Web Application Firewalls (WAF): Use WAFs to filter out malicious traffic before it reaches the application.	
References	
https://dev.to/carrie_luo1/how-does-waf-prevent-sql-injection-29gl https://www.picussecurity.com/resource/blog/waf-bypass-using-json-based-sql-injection-attacks	

Proof of Concept



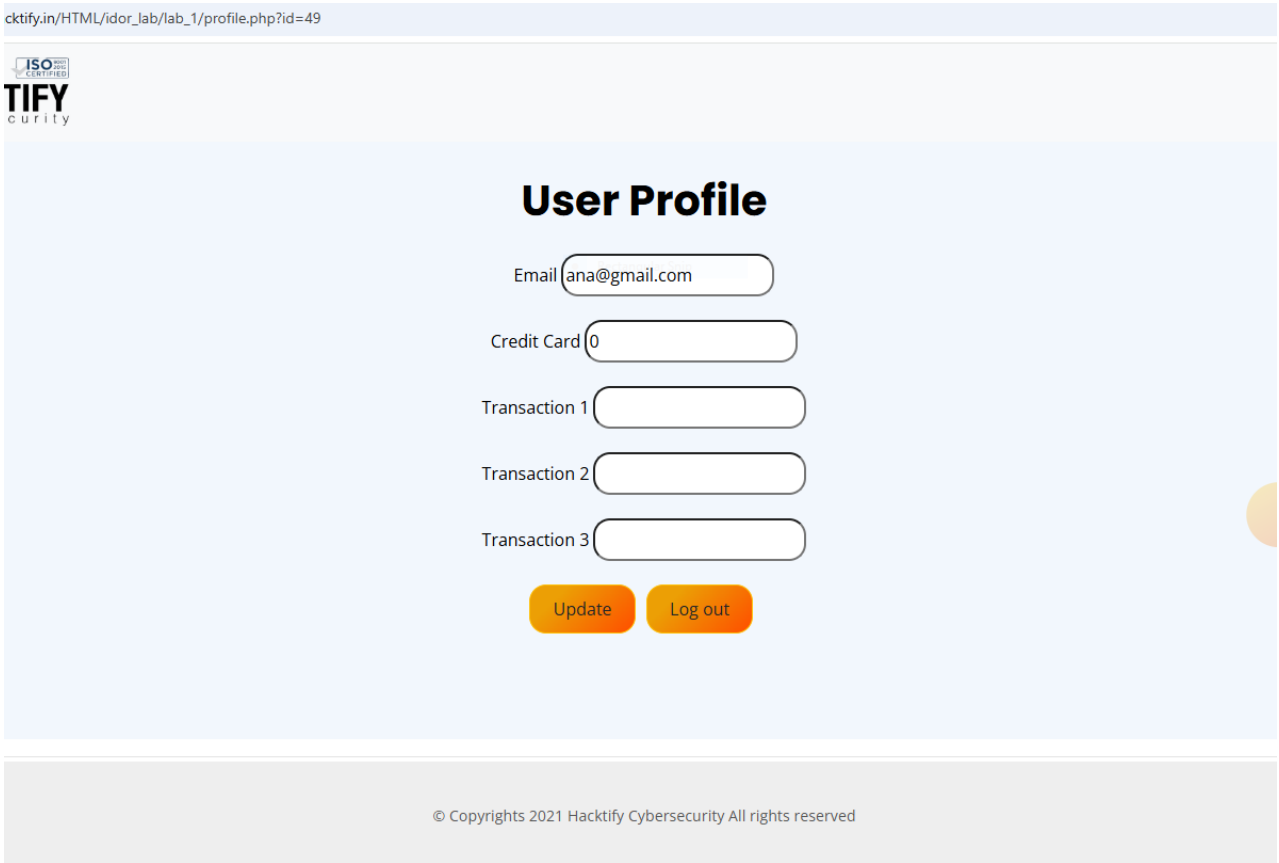
2. Insecure Direct Object References

2.1. Give Me My Amount!!

Reference	Risk Rating
Give Me My Amount!!	**Low**
Tools Used	
No tool is used.	
Vulnerability Description	
In the bank user login page of a website, an IDOR (Insecure Direct Object Reference) vulnerability exists, allowing unauthorized access to other users' accounts. This vulnerability occurs due to inadequate access controls and improper validation of user IDs. We exploit this vulnerability by intercepting and modifying the user ID parameter in the login request using tools like Burp Suite.	
How It Was Discovered	
Discovered through manual analysis by changing userid from 48 to 49.	
Vulnerable URLs	
https://labs.hacktify.in/HTML/idor_lab/lab_1/index.php	
Consequences of not Fixing the Issue	
Unauthorized access to sensitive financial information of users, leading to potential identity theft or fraud. Unauthorized transactions or changes to user account settings, causing financial loss or reputational damage to the bank.	
Suggested Countermeasures	
Implement proper access controls and authentication mechanisms to ensure that users can only access their own accounts.	

Use session tokens or other secure identifiers instead of directly exposing user IDs in URLs or requests
References
https://owasp.org/www-community/attacks/Insecure_Direct_Object_Reference https://portswigger.net/burp/documentation

Proof of Concept

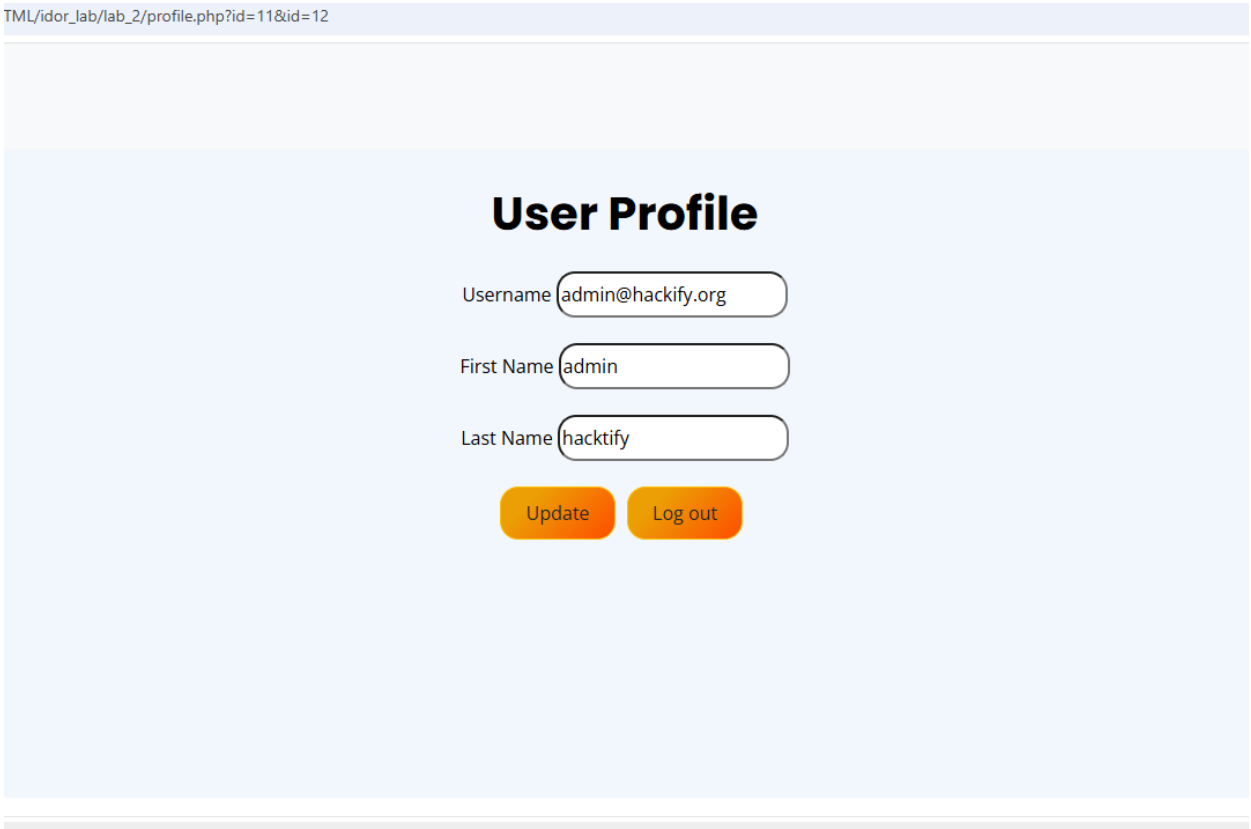


2.2. Stop Polluting My Params!

Reference	Risk Rating
Stop Polluting My Params!	**Medium**
Tools Used	
Manual analysis and no specific tools is used.	
Vulnerability Description	
In the user login page of a website, an IDOR (Insecure Direct Object Reference) vulnerability exists, enabling unauthorized access to other users' accounts by manually changing the user ID in the URL. This vulnerability arises due to inadequate access controls and improper validation of user IDs. We exploit this vulnerability by directly manipulating the user ID parameter in the URL to access other user's accounts without authentication.	
How It Was Discovered	

Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/idor_lab/lab_2/index.php
Consequences of not Fixing the Issue
Unauthorized access to sensitive financial information of users, leading to potential identity theft or fraud. Unauthorized transactions or changes to user account settings, causing financial loss or reputational damage to the bank
Suggested Countermeasures
Implement proper access controls and authentication mechanisms to ensure that users can only access their own accounts. Use session tokens or other secure identifiers instead of directly exposing user IDs in URLs or requests
References
https://www.varonis.com/blog/what-is-idor-insecure-direct-object-reference

Proof of Concept

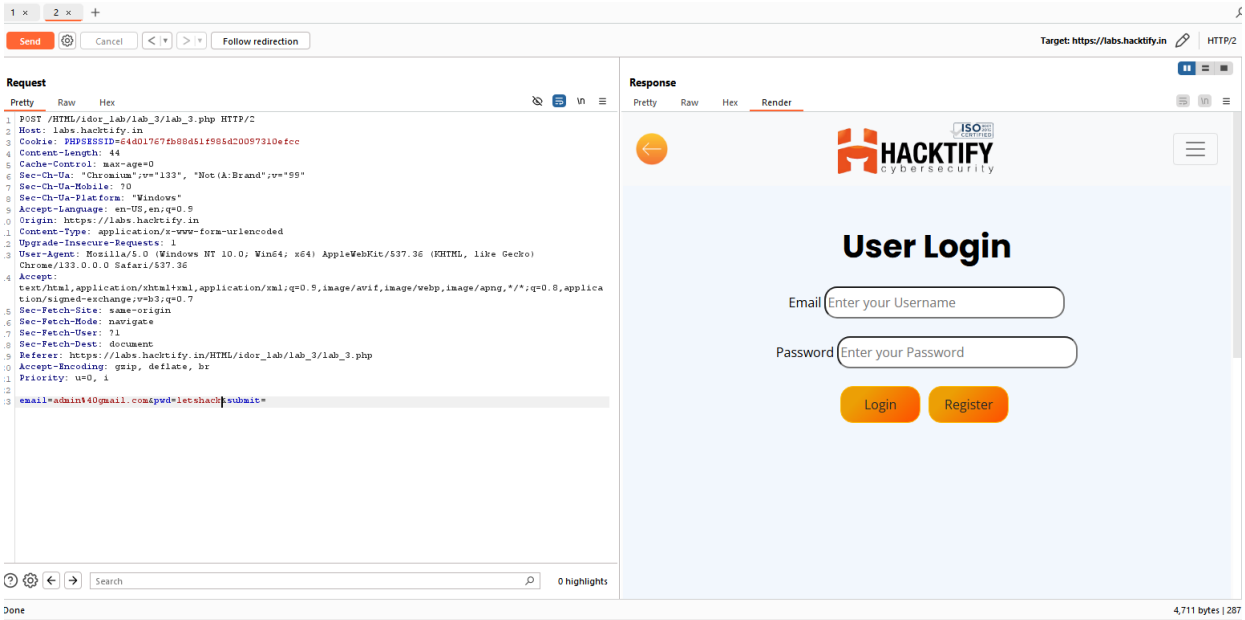


2.3. Someone Changed My Password!

Reference	Risk Rating
Someone Changed My Password!	**High**
Tools Used	
Manual and Burp suite.	

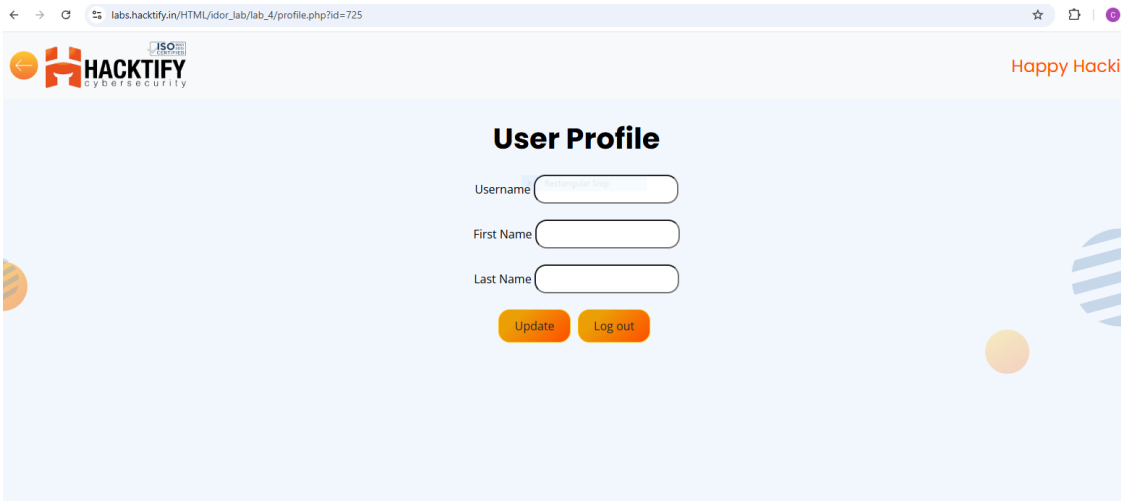
Vulnerability Description
Insecure Direct Object Reference (IDOR) allows unauthorized access to resources by manipulating object identifiers in URLs.
How It Was Discovered
Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/idor_lab/lab_3/index.php
Consequences of not Fixing the Issue
Potential unauthorized access to sensitive data, compromise of user accounts, and legal liabilities.
Suggested Countermeasures
Implement proper access controls, such as session-based access instead of direct object references, and perform input validation on user-supplied data.
References
https://www.scaler.com/topics/cyber-security/idor/

Proof of Concept



2.4. Change your Methods!

Reference	Risk Rating
Change your Methods!	**Medium**
Tools Used	
None, manual analysis was conducted.	
Vulnerability Description	
Insecure Direct Object Reference (IDOR) allows unauthorized access to resources by manipulating object identifiers in URLs.	
How It Was Discovered	
Manual Analysis	

Vulnerable URLs
https://labs.hacktify.in/HTML/idor_lab/lab_4/index.php
Consequences of not Fixing the Issue
Potential unauthorized access to sensitive data, compromise of user accounts, and legal liabilities.
Suggested Countermeasures
Implement proper access controls, such as session-based access instead of direct object references, and perform input validation on user-supplied data.
References
-NIL-

Proof of Concept: